

Robust Distortion-free Watermarks for Language Models

Rohith Kuditipudi John Thickstun Tatsunori Hashimoto Percy Liang

Department of Computer Science
Stanford University

July 2023

Abstract

We propose a methodology for planting watermarks in text from an autoregressive language model that are robust to perturbations without changing the distribution over text up to a certain maximum generation budget. We generate watermarked text by mapping a sequence of random numbers—which we compute using a randomized watermark key—to a sample from the language model. To detect watermarked text, any party who knows the key can align the text to the random number sequence. We instantiate our watermark methodology with two sampling schemes: inverse transform sampling and exponential minimum sampling. We apply these watermarks to three language models—OPT-1.3B, LLaMA-7B and Alpaca-7B—to experimentally validate their statistical power and robustness to various paraphrasing attacks. Notably, for both the OPT-1.3B and LLaMA-7B models, we find we can reliably detect watermarked text ($p \leq 0.01$) from 35 tokens even after corrupting between 40-50% of the tokens via random edits (i.e., substitutions, insertions or deletions). For the Alpaca-7B model, we conduct a case study on the feasibility of watermarking responses to typical user instructions. Due to the lower entropy of the responses, detection is more difficult: around 25% of the responses—whose median length is around 100 tokens—are detectable with $p \leq 0.01$, and the watermark is also less robust to certain automated paraphrasing attacks we implement.¹

1 Introduction

The ability of language models to mass produce human-like text creates an acute, renewed emphasis on the importance of provenance of generated content. For example, the website StackOverflow has banned users from posting answers using OpenAI’s ChatGPT model to mitigate the spread of misinformation on the platform [25]. A reliable forensic tool for attributing text to a particular language model would empower individuals—such as platform moderators and teachers—to enact and enforce policies on language model usage; it would also better enable model providers to track the (mis)use of their models, e.g., to scrub synthetic text from the training data of future language models.

To achieve provenance, a *watermark* is a signal embedded within some generated content—in our case, text from a language model—that encodes the source of the content. We consider a setting where an untrusted third party user queries a language model (LM) by sending prompts to a trusted provider (Figure 1): the LM provider generates text from their language model with a watermark so that a detector may later identify the source of the text if the user publishes it. The ideal watermark should satisfy at least the following three desiderata:

1. **distortion-free**—the watermark should preserve the original text distribution;
2. **agnostic**—it should be detectable without the language model and/or prompt;
3. **robust**—it should withstand perturbations of the watermarked text.

¹We release all code publicly at <https://github.com/jthickstun/watermark>.

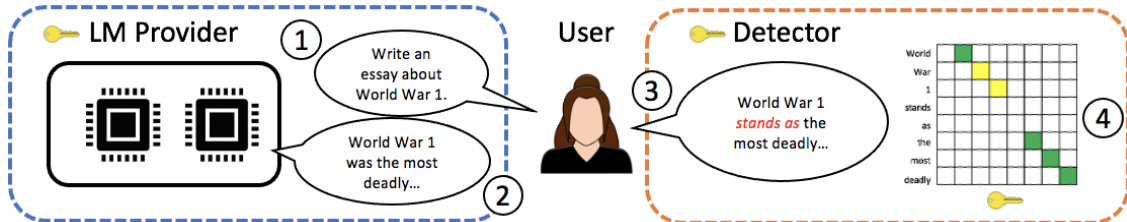


Figure 1: We define the following watermarking protocol between three parties: the LM provider, the user, the detector. The LM provider and the detector coordinate via a shared key, while the user is an untrusted third party. The protocol consists of four steps: 1) the user sends a prompt to the LM provider; 2) the LM provider generates watermarked text to the user; 3) the user edits the watermarked text (to avoid detection) and publishes the edited text; 4) the detector verifies which of the published text came from the LM provider.

Existing watermarks either distort the model’s sampling distribution, thus altering the API functionality [13, 1], or are not robust to editing or cropping the text [5]. Meanwhile, classical steganographic techniques for covertly encoding messages within samples of text from a language model are neither agnostic nor robust [30]. We develop the first watermarks for attributing text to a language model that achieve all three desiderata.

Our methodology consists of two components, which the LM provider and detector respectively use to execute the two steps of the protocol in Figure 1 under their control: a **generate** method that deterministically maps a sequence ξ of random numbers encoded by a watermark key—which we call the watermark key sequence—to a sample from the language model, and a **detect** method that aligns a putative watermarked text with the watermark key sequence using the shared key. Informally, our watermarks are *distortion-free* in the sense that—marginalizing over the watermark key sequence—each call to **generate** is equal in distribution to a sample from the original language model, i.e., the distribution

$$P(\mathbf{text}) = \int_{\xi} \mathbf{1}\{\mathbf{text} = \mathbf{generate}(\xi, \mathbf{prompt})\} d\nu(\xi)$$

is equal to the original language model’s sampling distribution.

The challenge of detecting watermarked text is that the detector cannot simply recompute **generate** and compare its output against the text since they do not necessarily know the prompt which produced the text: in practice, users often crop the prompt when publishing text from a language model. Our watermarks are *agnostic* in the sense that they are easily detectable with a suitable model-agnostic and prompt-agnostic test statistic ϕ such that $\phi(\mathbf{generate}(\xi, \mathbf{prompt}), \xi) \ll \phi(\mathbf{text}, \xi)$ for any $\mathbf{text}$ that is independent of the watermark key sequence. The idea here is that the detector may use ϕ within **detect** to compute a p -value with respect to the null hypothesis that the text is independent of the watermark key sequence, i.e., that the text is not watermarked.

To ensure **detect** is *robust* to edits of the watermarked text, the core idea underpinning the design of each test statistic ϕ is to leverage techniques for robust sequence alignment to align a putative watermarked text with the watermark key sequence; we quantify the quality of the alignment using an “alignment cost” specific to each watermark. The sequence alignment procedure ensures the watermark is detectable from even a small, corrupted block of watermarked text planted within some other larger text. Of course, a sufficiently motivated and/or sophisticated user can still evade detection by simply rewriting the text from scratch

themselves (or, using another language model to generate the text); the point of a robust watermark is simply that the amount of effort and/or resources a user requires to produce text that evades watermark detection should be commensurate to what they would have expended had they not had access to the watermarked language model in the first place.

Whereas **generate** is a deterministic function, if our watermark produced the same text every time for each prompt it would not be very useful. We resolve this limitation by designing a wrapper **shift-generate** around **generate** that calls **generate** using a randomly chosen subsequence of ξ instead of generating tokens from the same starting point each time. For the same reasons that **detect** is robust to editing and cropping watermarked text, calling **generate** in this fashion does not affect watermark detectability. In practice, the statistical power of our watermarks improves exponentially with respect to the length of the putative watermarked text and diminishes only linearly with the length of the random number sequence; thus, by increasing the length of the random number sequence, we can reduce the probability of reusing the same random subsequence while still ensuring our watermark has good statistical power (i.e., that it yields low p -values for watermarked text). So long as we do not reuse an element of the key sequence, successive calls to **shift-generate** will be jointly indistinguishable from regular calls to the language model.

To remark briefly on the work most closely related to ours, we contrast the distortion-free property of our watermarks with the hashing-based watermarks of Kirchenbauer et al. [13] and Aaronson [1] that bias the distribution of watermarked text towards certain k -grams by hashing a sliding window of the previous $k - 1$ tokens to determine the next token pseudorandomly. We give examples of prompts (e.g., “Give me a list of 20 movies.”) for which the bias due to hashing is clearly noticeable in our experiments. Christ et al. [5] propose a variation of hashing in which the window size changes based on the entropy of the generated tokens to avoid hash collisions with high probability. Their motivation is similar to ours in that they focus on preserving the original text distribution; however, like Kirchenbauer et al. [13] and Aaronson [1], using larger window sizes hurts robustness as an adversary can break the watermark by replacing a single token in each window. Our watermark is not only distortion-free but also robust to substantial corruption of the text, which is crucial in practice. We defer a more thorough discussion of related work to the next section (Section 1.1).

We describe the details of our methodology in Section 2, wherein we give two instantiations of watermarks—using inverse transform sampling and exponential minimum sampling—and provide analyses of their statistical power. We experimentally validate the power and robustness of our watermarks using the OPT-1.3B, LLaMA-7B and Alpaca-7B language models in Section 3. Across all models, we find the second instantiation using exponential minimum sampling to be the most powerful. For both the OPT-1.3B and LLaMA-7B models, using this watermark we can reliably detect watermarked text ($p \leq 0.01$) from 35 tokens even after corrupting between 40-50% of the tokens via random edits (i.e., substitutions, insertions or deletions); the watermark also remains detectable from 50 tokens even after paraphrasing the text by translating to French/Russian and back. For the Alpaca-7B model, we conduct a case study on the feasibility of watermarking responses to typical user instructions. Due to the lower entropy of the responses, detection is more difficult: around 25% of the responses—whose median length is around 100 tokens—are detectable with $p \leq 0.01$, and the watermark is also less robust to paraphrasing. We release code for implementing the watermark and reproducing the experiments in this paper, as well as additional supplementary material including an in-browser demo of the watermark detector.²

²For assets and supplemental material, see: <https://github.com/jthickstun/watermark>.

1.1 Related work

Text watermarking is a special case of linguistic steganography, in that the goal is to convey a hidden message—the watermark—within a passage of text. Existing approaches to linguistic steganography fall under two broad categories: *edit-based* methods that modify a pre-existing text, and *generative* methods that construct a distribution over cover text [23]. Crucially, in contrast to steganography, the literature on digital watermarking has historically foregrounded robustness to corruption as a key attribute of a good watermark [12, 3]. In this light, a text watermark should be able to withstand some perturbations of the text, thus precluding the direct application of many existing techniques for linguistic steganography [6, 30, 18].

Older work on text watermarking considers editing a pre-existing text to include a watermark [17, 2, 27]; for a survey of edit-based watermarks, see Kamaruddin et al. [11]. In contrast, we are interested in generating watermarked text while preserving the distribution over the text from a language model. Work on generative watermarking is nascent, underwritten by recent advances in open-ended text generation [4]. Pioneering work by Venugopal et al. [24] proposed a generative watermark for the output of a machine translation system, biasing the system towards translations with particular features that can later be detected using a hypothesis test.

Our work is most closely related to Kirchenbauer et al. [13], who watermark text from a language model by reweighting the token log-probabilities from the model at inference time as a function (i.e., hash) of the previous $k-1$ tokens, where $k \in \mathbb{N}$ is a hyperparameter. In ongoing unpublished work concurrent to ours, Aaronson [1] describes a technique for watermarking language models using exponential minimum sampling (a close relative of the Gumbel trick [15]) to sample from the model, where the inputs to the sampling mechanism are also a hash of the previous $k-1$ tokens. Neither watermark is distortion-free, and in fact we show in our experiments that the distortions manifest noticeably in practice (e.g., excessive repetition of certain tokens). Specifically, both Kirchenbauer et al. [13] and Aaronson [1] bias the distribution toward a subset of k -grams. Increasing k makes the bias less noticeable but hurts the robustness of both watermarks: an adversary can break the signal from a particular token by replacing any one of the previous $k-1$ tokens.

Also concurrent to our work, Christ et al. [5] propose watermarking blocks of text from a language model by hashing each block to seed a sampler for the next block. Christ et al. [5] vary their block sizes—which are analogous to the hyperparameter k of Kirchenbauer et al. [13] and Aaronson [1]—as a function of the empirical entropy of the constituent tokens to avoid using the same seed twice with high probability. Their work is similar to ours in that they preserve the original language model’s sampling distribution; however, the resulting watermark is not robust since in order to mitigate the distortion induced by hashing the block sizes must be sufficiently large to avoid hash collisions with high probability over all blocks and—similar to Kirchenbauer et al. [13] and Aaronson [1]—replacing any token in the previous block breaks the watermark in the next block. Whereas Christ et al. [5]—who do not run experiments—choose their block sizes to be sufficiently large to minimize distortion, Kirchenbauer et al. [13] and Aaronson [1] recommend choosing k to be a small constant in practice, which ensures a moderate amount of robustness by introducing some distortion. Finally, whereas our definition distortion-freeness implies exact equality in distribution of watermarked text to unwatermarked text for a single query to the language model, Christ et al. [5] propose a definition of “undetectability” that implies approximate equality in distribution, i.e., approximate distortion-freeness, over multiple queries. Using **shift-generate**, we also achieve approximate distortion-freeness in the multiple query setting, though the runtime of

our watermark detection procedure must grow with the number of queries; we discuss these trade-offs in more detail in Section 4.

An alternative approach for detecting synthetic text is to learn a classifier between synthetic and human text [10, 14]. A key advantage of such methods over watermarking is that they do not require coordination with the original producer of the text (i.e., the LM provider); however, their effectiveness is distribution dependent and they do not provide a priori (distribution-free) guarantees on the significance level of detection (i.e., Type I errors).

Finally, we note that our setting is different from the literature on planting watermarks in the training data of machine learning models, e.g., to infer the model’s training set or otherwise influence the model’s output [8, 9, 29]. Such watermarks are not distortion-free by design, since the point is to plant some learnable signal in the training data that influences the behavior of models which train on the watermarked data.

2 Methodology and theoretical analysis

Let $\mathcal{V}$ be a discrete set, i.e., the *vocabulary*, and let $p \in \mathcal{V}^* \rightarrow \Delta(\mathcal{V})$ be an autoregressive *language model* which maps a string of arbitrary length to a distribution over the vocabulary, with $p(\cdot | x)$ denoting the distribution of the next token given the prefix $x \in \mathcal{V}^*$. Let Ξ denote the space in which the elements of the watermark key sequence lie. Recall the main protocol (Figure 1) which defines our problem setting:

0. The LM provider shares a random watermark key sequence $\xi \in \Xi^*$ with the detector;
1. The user sends a prompt $x \in \mathcal{V}^*$ to the LM provider;
2. The LM provider generates text $Y \in \mathcal{V}^*$ by $Y = \text{generate}(x, \xi)$;
3. The user publishes text $\tilde{Y} \in \mathcal{V}^*$, which may be either (i) (an edited version of) the generated text Y or (ii) text independent of Y (e.g., text that they wrote themselves);
4. The detector determines if $\tilde{Y}$ is watermarked—i.e., if $\tilde{Y}$ depends on the watermark key sequence—by computing a p -value $\hat{p} = \text{detect}(\tilde{Y}, \xi)$ with respect to the null hypothesis that $\tilde{Y}$ is independent of ξ (i.e., not watermarked).

2.1 Protocol details

In the protocol, the LM provider calls the **generate** method (Algorithm 1) to autoregressively generate text from a language model using a *decoder* function $\Gamma : \Xi \times \Delta(\mathcal{V}) \rightarrow \mathcal{V}$ which maps an element ξ_i of the watermark key and a distribution over the next token to a next token prediction. By design, over the randomness of ξ_i the prediction should constitute a sample from the distribution, i.e., $\mathbb{P}(\Gamma(\xi_i, \mu) = y_i) = \mu(y_i)$.

Definition 1. A decoder $\Gamma : \Xi \times \Delta(\mathcal{V}) \rightarrow \mathcal{V}$ is *distortion-free* with respect to (the distribution of) a random variable $\xi \in \Xi$ if for any $\mu \in \Delta(\mathcal{V})$ and $y \in \mathcal{V}$ it satisfies $\mathbb{P}(\Gamma(\xi, \mu) = y) = \mu(y)$.

We relate Definition 1 to our informal definition of distortion-free text in the introduction through the following simple lemma. Essentially, so long as the watermark key sequence is long enough that we do not reuse any part of it to generate text, the only material difference between an LM provider using **generate** versus sampling directly from the language model is that the sequence ξ is an input to the method rather than resampled i.i.d. within the method

for each call. We treat the language model p , the decoder Γ , and generation length m as internal parameters of the **generate** method.

Lemma 2.1. *Let $m, n \in \mathbb{N}$ with $n \geq m$. Let Γ be distortion free with respect to a distribution $\nu \in \Delta(\Xi)$ and let $\{\xi_i\}_{i=1}^n \stackrel{i.i.d.}{\sim} \nu$. Let $Y = \text{generate}(\xi; m, p, \Gamma)$. Then $Y_i \sim p(\cdot \mid Y_{:i-1})$ for $i \in [m]$.*

Proof. As $n \geq m$, we have $\{\xi_i\}_{i=1}^m \stackrel{i.i.d.}{\sim} \nu$. The claim then follows immediately from applying Definition 1 to Line 2 of **generate** for $i \in [m]$. $\square$

To simplify the remainder of the presentation, we do not pass a prompt as input to **generate**. As the language model p is arbitrary and **detect** is model-agnostic, this simplification is without loss of generality since p itself may model the distribution of text from some base model given an arbitrary prompt. Also, unless stated otherwise, without loss of generality we let $\mathcal{V} = [N]$ throughout the paper, where $N \in \mathbb{N}$ is the vocabulary size.

Algorithm 1: Watermarked text generation (**generate**)

Input : watermark key sequence $\xi \in \Xi^n$
Params: generation length m , language model p , decoder Γ
Output: string $y \in \mathcal{V}^m$
1 **for** $i \in 1, \dots, m$ **do**
2 $y_i \leftarrow \Gamma(\xi_{i \% n}, p(\cdot \mid y_{:i-1}))$
3 **return** y

The detector calls the **detect** method (Algorithm 2) to compute—via a permutation test with T resamples—a p -value with respect to a test statistic $\phi : \mathcal{V}^* \times \Xi^* \rightarrow \mathbb{R}$ for the null hypothesis that $\tilde{Y}$ is not watermarked, i.e., that $\tilde{Y}$ is independent of ξ . The output $\hat{p}$ of **detect** is a proper non-asymptotic p -value: if $\tilde{Y}$ is not watermarked, then each $(\tilde{Y}, \xi^{(t)})$ constitutes an independent, identically distributed copy of $(\tilde{Y}, \xi)$ and therefore by symmetry $\hat{p}$ is uniformly distributed over $\{1/(T+1), 2/(T+1), \dots, 1\}$ for any (non-atomic) test statistic.³ If ϕ returns a small p -value (e.g., 0.0001) then the text is likely watermarked; if the p -value is large (e.g., 0.25), then the text might not be.

The goal then is to design the test statistic ϕ (Algorithm 3) such that $\hat{p}$ will typically be small if $\tilde{Y}$ is watermarked. In particular, the goal is to identify an alignment cost $d : (\mathcal{V} \times \Xi)^* \rightarrow \mathbb{R}$, which measures the quality of a match between a subsequence of the input text and a subsequence of the watermark key, and use this to define ϕ as the minimum cost alignment between length k subsequences of the text and key.

This alignment-based detection strategy makes the watermark robust, since even if the user crops or otherwise corrupts Y , a single block of preserved watermarked text within some larger body of unwatermarked text will suffice to trigger a low p -value from **detect**. The actual form of the alignment cost will be specific to each watermark—in particular, it will depend on the nature of the decoder Γ in **generate**. Our most robust watermarks incorporate a soft notion of edit distance (i.e., Levenshtein distance) into the computation of the alignment cost via dynamic programming, with runtime scaling quadratically in the block size. Thus,

³By non-atomic, we mean for any $c \in \mathbb{R}$ that $\mathbb{P}(\phi(Y, \xi) = c) = 0$ so that almost surely we will not have to break ties (meaning, if $\phi(y, \xi) = \phi_t$) when computing $\hat{p}$. In case of ties (i.e., if the test statistic is atomic), we can either modify **detect** to break ties uniformly at random, or simply report valid but conservative p -values by leaving **detect** as is.

Algorithm 2: Watermarked text detection (**detect**)

Input : string $y \in \mathcal{V}^*$, watermark key sequence $\xi \in \Xi^n$
Params: test statistic ϕ ; watermark key sequence distribution $\nu \in \Delta(\Xi^n)$; resample size T
Output: p-value $\hat{p} \in [0, 1]$

```
1 for  $t \in 1, \dots, T$  do
2    $\xi^{(t)} \sim \nu$ 
3    $\phi_t \leftarrow \phi(y, \xi^{(t)})$ 
4  $\hat{p} \leftarrow \frac{1}{T+1} \left( 1 + \sum_{t=1}^T \mathbf{1}\{\phi_t \leq \phi(y, \xi)\} \right)$ 
5 return  $\hat{p}$ 
```

letting m be the length of the input text y , n be the length of the watermark key sequence ξ , and k be the block size, the cost of computing the test statistic is $O(mnk^2)$.

Algorithm 3: Test statistic (ϕ)

Input : string $y \in \mathcal{V}^*$, watermark key sequence $\xi \in \Xi^n$
Params: alignment cost d , block size k
Output: test statistic value $\phi(y, \xi) \in \mathbb{R}$

```
1 for  $i \in 1, \dots, \text{len}(y) - k + 1$  do
2   for  $j \in 1, \dots, n$  do
3      $y^i \leftarrow \{y_{i+\ell}\}_{\ell=0}^{k-1}$ ,  $\xi^j \leftarrow \{\xi_{(j+\ell)\%n}\}_{\ell=0}^{k-1}$ 
4      $\hat{d}_{i,j} \leftarrow d(y^i, \xi^j)$ 
5 return  $\min_{i,j} \hat{d}_{i,j}$ 
```

To illustrate how the decoder and the alignment cost fit together, we give a simple example for the toy setting of a binary vocabulary.

Example 1: Consider a binary vocabulary $\mathcal{V} = \{0, 1\}$. To generate $Y \in \{0, 1\}^*$ from the model, the LM provider shares $\{\xi_i\}_{i=1}^n \stackrel{\text{i.i.d.}}{\sim} \text{Unif}([0, 1])$ with the detector and let $Y_i = 0$ if $\xi_i \leq p(0 \mid Y_{i-1})$ and $Y_i = 1$ otherwise. In particular, defining the decoder Γ by

$$\Gamma(\xi_i, \mu) := \begin{cases} 0 & \xi_i \leq \mu(0) \\ 1 & \xi_i > \mu(0), \end{cases}$$

let $Y = \text{generate}(\xi; m, p, \Gamma)$ for some $m \leq n$. Then Y is a valid sample from the language model as $\mathbb{P}(\xi_i \leq p(0 \mid Y_{i-1})) = p(0 \mid Y_{i-1})$, and crucially Y and ξ are correlated (i.e., if ξ_i is sufficiently close to zero then $Y_i = 0$, and likewise if ξ_i is sufficiently close to one then $Y_i = 1$). Thus, we can define the alignment cost $d(y, \xi) = \|y - \xi\|_1$.

Assuming for the sake of this example that $n = m$ and the user does not corrupt the watermarked text from the LM provider, i.e., $\tilde{Y} = Y$, the detector can run **detect** to verify that $\tilde{Y}$ is watermarked using the test statistic ϕ with alignment cost d and block size $k = m$. The value of the test statistic will then be at most the ℓ_1 norm of $\tilde{Y} - \xi$. $\diamond$

2.2 Handling multiple queries

In the above example, the LM provider generates the same text each time from the watermark key sequence, which is not ideal in practice. One solution for avoiding reusing elements of the watermark key sequence across queries is to make **generate** stateful, thus enabling the LM provider to generate a total of $\lfloor n/m \rfloor$ independent watermarked text samples of m tokens each from the language model. Instead, to avoid persisting state, we provide a randomized wrapper **shift-generate** (Algorithm 4) around **generate** and modify the watermarking protocol from the start of the section to allow the LM provider to call the **shift-generate** instead of **generate** in the second step of the protocol. The wrapper **shift-generate** randomly shifts the watermark key sequence before passing the shifted sequence to **generate**. Shifting the watermark key sequence does not affect the value of the test statistic in **detect**, since to compute the test statistic the detector anyways searches over all subsequences of the watermark key sequence to find the best match for each block of text. There are n possible shifts, each of which may produce a distinct text; while in principle these n texts will correlate with each other due to sharing elements of the watermark key sequence, in practice we find the effects of these correlations are not noticeable. The so-called birthday paradox [7] implies the LM provider can typically expect to call **shift-generate** $\Omega(\sqrt{n})$ times, each time producing a different text, before reusing the same offset twice. In fact, the provider can expect call **shift-generate** $\Omega(\sqrt{n/m})$ times before reusing a subsequence, in which case the constituent $\Omega(\sqrt{nm})$ tokens in these texts will be indistinguishable from regular samples from the language model.

In general, we can bound the distortion (i.e., statistical distance from regular samples) of **shift-generate** in the multi-query setting by the probability of reusing an element of the watermark key sequence. Specifically, for T queries and a maximum generation length of m tokens per query, we will achieve negligible, i.e., $o(1)$ distortion, so long as $n = \omega(mT^2)$. Thus, similar to Christ et al. [5] we can achieve approximate distortion-freeness in the multi-query setting. However, unlike Christ et al. [5], to achieve approximate distortion-freeness in this setting the computational cost of our watermark detection procedure must grow with the target number of queries. In principle, this trade-off between the degree of distortion and the runtime of watermark detection means at least in an asymptotic sense that the latter effectively upper bounds the number of queries to the LM provider an attacker would require to learn information about the watermark key sequence. In practice, we expect the cost per token of queries to the LM provider will be significant enough to make such attacks expensive to implement.

Algorithm 4: Randomized watermarked text generation (**shift-generate**)

Input : watermark key sequence $\xi \in \Xi^n$
Params: generation length m , language model p , decoder Γ
Output: string $y \in \mathcal{V}^m$
1 $\tau \sim \text{Unif}([n])$, $\xi' \leftarrow \{\xi_{(i+\tau)\%n}\}_{i=1}^m$
2 **return** **generate**($\xi'; m, p, \Gamma$)

2.3 Terminology: watermark strategies and watermark potential

Henceforth, we use the term *watermarking strategy* to refer to a concrete instantiation of the **shift-generate**, **generate** and **detect** methods by specifying the internal parameters of both

algorithms (i.e., the decoder Γ , the test statistic ϕ and the watermark key sequence distribution ν). We give concrete watermarking strategies in the following sections (Sections 2.4 and 2.5). For each watermarking strategy, we show two main results: we prove the decoder is distortion-free and also obtain high probability upper bounds on the p -values of watermarked text—as a function of the length of the text and the watermark key sequence. We emphasize that only the former result (i.e., that the decoder is distortion-free) is critical to the validity of our main claims; we intend the latter collection of results to provide intuition for when we would expect the detector to have sufficient power and to anticipate the forthcoming experimental results in Section 3. The strength of the p -value upper bounds will depend on the observed token probabilities of (watermarked) text, through a quantity which we evocatively term the *watermark potential*.

Definition 2. (watermark potential) Define $\alpha : \mathcal{V}^* \rightarrow \mathbb{R}$ by

$$\alpha(y) := 1 - \frac{1}{\text{len}(y)} \sum_{i=1}^{\text{len}(y)} p(y_i \mid y_{:i-1}).$$

The watermark potential of text from a deterministic language model will always be zero, whereas for a high-entropy model it will approach one. The degree to which it is possible for the detector to reliably distinguish watermarked text from unwatermarked text necessarily depends on the watermark potential of the LM provider’s language model. For example, if the language model is deterministic, then any distortion-free watermark will necessarily have zero statistical power. We formalize this intuition by establishing the following general lower bound on the detection accuracy of any watermarking strategy as a function of the watermark potential of the original language model. In particular, we lower bound the error of any classifier $h : \mathcal{V}^* \times \Xi^* \rightarrow \{-1, +1\}$ that tries to distinguish watermarked (positive label) versus nonwatermarked text (negative label) given some watermark key ξ (we make no assumption on the distribution of ξ except that it is independent of unwatermarked text by definition). We defer the proof of Lemma 2.2 to Appendix A.

Lemma 2.2. *Let $Y'_i \sim p(\cdot \mid Y'_{:i-1})$ for $i \in [m]$. Let $Y \stackrel{d}{=} Y'$ and let $\xi \in \Xi^*$ be a random variable that is independent of Y' . Let $h : \mathcal{V}^* \times \Xi^* \rightarrow \{-1, +1\}$ be a classifier. Let $c > 0$ and define the set $\mathcal{V}_c \subset \mathcal{V}^m$ by*

$$\mathcal{V}_c := \{y : p(y_i \mid y_{:i-1}) \geq \exp(-c/2) \text{ for all } i \in [m]\}.$$

Then

$$\mathbb{P}(h(Y, \xi) = -1) + \mathbb{P}(h(Y', \xi) = 1) \geq \mathbb{E}[\exp(-cm\alpha(Y)) \mathbf{1}\{Y \in \mathcal{V}_c\}].$$

Lemma 2.2 implies it is impossible to test between any watermarked and non-watermarked text (i.e., between Y versus Y') that are equal in distribution (i.e., distortion-free) if the text typically has low watermark potential, irrespective of the design of the watermark key; in particular, the sum of the Type I and II (resp., false positive/negative) error rates of h will be close to one if the watermark potential is close to zero. The theorem is not tight: depending on the language model, its result may be vacuous for small values of c (e.g., the constants which appear in our upper bounds) since only texts whose token likelihoods all exceed $\exp(-c/2)$ contribute to the lower bound. Also our upper bounds scale inverse exponentially with the square of the watermark potential, which will always be smaller than the watermark potential itself since the watermark potential is bounded between zero and one.

The point of the forthcoming p -value upper bounds for the watermarking strategies in Sections 2.4 and 2.5 is to establish the existence of test statistics for each watermark such that the statistical power of the watermark improves exponentially with the length of the text and decays at most linearly with the length of the watermark key sequence. The test statistics we use to prove these upper bounds differ slightly from those we employ in our experiments: in the former case, we prioritize the simplicity of stating the bounds in terms of watermark potential, whereas in the latter case, we prioritize empirical performance.

2.4 Watermarking via inverse transform sampling

Inverse transform sampling is a general technique for sampling from a univariate distribution by taking the pushforward of a uniform random variable through its inverse cumulative distribution function (CDF). Crucially, the technique is valid irrespective of the ordering of the CDF, a property which we presently leverage to construct a watermarking strategy in which **generate** is distortion-free and also **detect** is agnostic. In particular, we implement **generate** with a decoder that maps a sequence of uniform random variables and permutations to tokens using inverse transform sampling. To detect watermarked text, the detector correlates the sequence of permuted indices of the tokens in the text with the sequence of uniform random variables to detect watermarked text. Meanwhile, for any nonwatermarked text, the sequence of permuted token indices will be i.i.d. uniform irrespective of the text itself and thus not correlate with the sequence of uniform random variables.

Formally, with Π as the space of permutations over the vocabulary $[N]$, for $\xi = (u, \pi) \in [0, 1] \times \Pi =: \Xi$ and any distribution $\mu \in \Delta([N])$, define the decoder by

$$\Gamma(\xi, \mu) := \pi^{-1}(\min\{\pi(i) : \mu(\{j : \pi(j) \leq \pi(i)\}) \geq u\}), \quad (1)$$

i.e., $\Gamma(\xi, \mu)$ is the token with the smallest index in the permutation π such that CDF of μ with respect to π is at least u . Generalizing the intuition from Example 1, we show this decoder is distortion-free in the following theorem.

Theorem 1. *Define Γ by equation (1). Let $\pi \in \Pi$ be arbitrary and let $U \sim \text{Unif}([0, 1])$, with $\xi := (U, \pi)$. Then Γ is distortion-free with respect to ξ .*

Proof. Recalling Definition 1, the result follows from showing for any $\mu \in \Delta([N])$ and $y \in [N]$ that $\mathbb{P}(\Gamma(\mu, \xi) = y) = \mu(y)$. To this end, by equation (1), we have $\Gamma(\mu, \xi) = y$ if and only if U lies in the interval

$$[\mu(\{y' : \pi(y') < \pi(y)\}), \mu(\{y' : \pi(y') \leq \pi(y)\})].$$

As the width of this interval is exactly $\mu(y)$, the result follows immediately. $\square$

Having shown that the decoder is distortion-free, we now proceed to analyze the detectability of the watermark. For convenience, define the normalization $\eta : [N] \rightarrow [0, 1]$ by $\eta(i) := (i - 1)/(N - 1)$. Analogous to the toy example, the sequences $\{\eta(\pi_i(Y_i))\}_{i=1}^m$ and U are correlated. Thus, for the sake of analysis, we define alignment cost $d : (\mathcal{V} \times \Xi)^* \rightarrow \mathbb{R}$ by

$$d(y, (u, \pi)) := - \sum_{i=1}^{\text{len}(y)} (u_i - 1/2) \cdot (\eta(\pi_i(y_i)) - 1/2), \quad (2)$$

i.e., the negative covariance (each U_i and $\eta(\pi_i(Y_i))$ both have expectation $1/2$).

We exactly characterize in Lemma 2.3 the difference in the expected value of our alignment cost on some text assuming the text is watermarked (i.e., generated using the same key as the detector) versus not watermarked in terms of the watermark potential of the text (Definition 2). To state the result, we define the constant $C_0 := \text{Var}(\eta(\text{Unif}([N])))$, where we abuse notation slightly to temporarily treat η as a pushforward map over distributions.⁴ We defer the proof of Lemma 2.3 to Appendix B.

Lemma 2.3. *Let $m, n \in \mathbb{N}$ with $n \geq m$, where m is the generation length and n is the watermark key length. Define the decoder Γ by equation (1) and the alignment cost d by equation (2). Let $\xi, \xi' \stackrel{i.i.d.}{\sim} \text{Unif}(\Xi^n)$ with $Y = \text{generate}(\xi; m, p, \Gamma)$. Then almost surely for all $i \in [m]$ and $j \in [n]$ we have*

$$\mathbb{E}[d(Y_i, \xi'_j) - d(Y_i, \xi_i) \mid Y] = C_0 \cdot (1 - p(Y_i \mid Y_{1:i-1})) = C_0 \alpha(Y_{1:i-1}).$$

Summing the result of Lemma 2.3 over $i \in [m]$ implies for any $j \in [n]$ that

$$\mathbb{E}[d(Y, \xi'_{(j+1:j+m)\%n}) - d(Y, \xi_{1:m}) \mid Y] = C_0 m \alpha(Y).$$

Thus, we can upper bound the p -value output by **detect** in Lemma 2.4 using a standard concentration argument and taking a union bound over $j \in [n]$. We defer the proof of Lemma 2.4 to Appendix B. In fact, we actually prove a more general result for $k \leq m$ wherein we allow $\tilde{Y}$ to be a subsequence of Y which the user may choose adaptively. We defer this more general result to Appendix B as it is more cumbersome to state.

Lemma 2.4. *Let $m, n \in \mathbb{N}$ with $n \geq m$, where m is the generation length and n is the watermark key length. Define the decoder Γ by equation (1), alignment cost d by equation (2), and ϕ by Algorithm 3 with block size $k = m$. Let $\xi, \xi' \stackrel{i.i.d.}{\sim} \text{Unif}(\Xi^n)$ with $Y = \text{generate}(\xi; n, p, \Gamma)$ and $\tilde{Y} = Y$. Then almost surely*

$$\mathbb{P}(\phi(\tilde{Y}, \xi') \leq \phi(\tilde{Y}, \xi) \mid \tilde{Y}) \leq 2n \exp(-kC_0^2 \alpha(\tilde{Y})^2/2).$$

Lemma 2.4 implies that with high probability the value of the test statistic on watermarked text with the correct key will be lower than with a resampled key. In particular, ignoring discretization errors due to the finite number of resamples T in **detect**, the lemma implies watermarked samples with watermark potential bounded away from zero (i.e., if the language model is not effectively deterministic) will have exponentially small expected p -values with respect to the length m of the text. The bound grows only linearly with the length n of the random number sequence, implying for moderately large m (e.g., $m = 50$) an LM provider can generate plenty of distortion-free watermarked text (i.e., $n = 2^{\Omega(m)}$ total tokens) while still enabling detection of the watermark from snippets of m tokens (e.g., 50 tokens typically amount to a couple sentences of text). Of course, recall the computational complexity of detection scales linearly with n , which in practice may be a more relevant limitation than the statistical power of the watermark.⁵

2.4.1 Robustness to substitutions, insertions and deletions

We show in Lemma 2.5 an analogous result to Lemma 2.4 holds even if an adversary corrupts the original watermarked text by substituting tokens. To state the lemma, we introduce a

⁴Note that $C_0 = \text{Var}(\text{Unif}([0, 1])) + o_N(1) = 1/12 + o_N(1)$.

⁵Note that both **detect** and the test statistic (Algorithm 3) are easily parallelizeable.

quantity $\tilde{\alpha}$ which depends on both the corrupted and original watermarked text and accounts for the decrease in the expected value of the test statistic (which recall for the original text is equal up to a numerical constant to the watermark potential of the text) due to token substitutions. We defer the proof of Lemma 2.5 to Appendix B.

Lemma 2.5. *Let $m, n \in \mathbb{N}$ with $n \geq m$, where m is the generation length and n is the watermark key length. Define the decoder Γ by equation (1), alignment cost d by equation (2), and ϕ by Algorithm 3 with $k = m$. Let $\xi, \xi' \stackrel{i.i.d.}{\sim} \text{Unif}(\Xi^n)$ with $Y = \text{generate}(\xi; m, p, \Gamma)$ and let $\tilde{Y} \in \mathcal{V}^m$ be conditionally independent of ξ and ξ' given Y . Define*

$$\tilde{\alpha}(y, \tilde{y}) := \frac{1}{\text{len}(y)} \sum_{i=1}^{\text{len}(y)} \mathbf{1}\{y_i = \tilde{y}_i\} (1 - p(y_i \mid y_{:i-1})) - \mathbf{1}\{y_i \neq \tilde{y}_i\} \frac{1}{N-1}.$$

Then almost surely

$$\mathbb{P}(\phi(\tilde{Y}, \xi') \leq \phi(\tilde{Y}, \xi) \mid Y, \tilde{Y}) \leq 2n \exp(-kC_0^2 \tilde{\alpha}(Y, \tilde{Y})^2/2).$$

Lemma 2.5 implies that even if an adversary replaces the vast majority of tokens in a watermarked text, detection with low p -values will still be possible so long as the remaining tokens have watermark potential bounded away from zero. In particular, the permuted indices of the original tokens will still positively correlate with the corresponding uniform random variables from the watermark key sequence, while those of the substituted tokens will exhibit a small negative correlation scaling as $O(1/N)$.

To handle insertions and deletions, we can robustify our test statistic by incorporating a soft notion of edit distance into our original alignment cost. The parameter γ in Definition 3 assigns a cost to each insertion and deletion operation when aligning the tokens y with the sequence ξ , while the base alignment cost d_0 defines the quality of the alignment via a cost function over substitutions. In practice, we drop the minimizations over $y' \in \mathcal{V}$ and $\xi' \in \Xi$ in the second and third cases respectively of the definition; we include them here to make our subsequent theoretical analysis cleaner.

Definition 3. (Levenshtein cost) Let $\gamma \in \mathbb{R}$ and $d_0 : \mathcal{V} \times \Xi \rightarrow \mathbb{R}$. For $y \in \mathcal{V}^*$ and $\xi \in \Xi^*$, define the *Levenshtein cost* $d_\gamma : \mathcal{V}^* \times \Xi^* \rightarrow \mathbb{R}$ by

$$d_\gamma(y, \xi) := \min \begin{cases} d_\gamma(y_{2:}, \xi_{2:}) + d_0(y_1, \xi_1) \\ d_\gamma(y, \xi_{2:}) + \min_{y' \in \mathcal{V}} d_0(y', \xi_1) + \gamma \\ d_\gamma(y_{2:}, \xi) + \min_{\xi' \in \Xi} d_0(y_1, \xi') + \gamma, \end{cases}$$

with $d_\gamma(y, (u, \pi)) := \gamma \cdot \text{len}(y)$ if ξ is empty and vice versa (as base cases).⁶

Redefining the test statistic ϕ using d_γ as the alignment cost—using d_0 from equation (2)—ensures **detect** is robust not only to substituting tokens, but also inserting and deleting tokens from watermarked text, as we show in Lemma 2.6. We defer the proof of Lemma 2.6 to Appendix B. To state the lemma, we first recursively define a notion of edit distance between two strings. The definition is equivalent to the minimum number of insertion and/or deletion operations needed to transform one string into the other (see Lemma B.2).

⁶For $y \in \mathcal{V}^*$ (resp., $\xi \in \Xi^*$), we let $y_{\text{len}(y)+1}$: (resp., $\xi_{\text{len}(\xi)+1}$) denote the empty string/sequence.

Definition 4. (edit distance) For $y, \tilde{y} \in \mathcal{V}^*$, define the *edit distance* by

$$d_{\text{edit}}(y, \tilde{y}) := \begin{cases} d_{\text{edit}}(y_2, \tilde{y}_2) & y_1 = \tilde{y}_1 \\ 1 + \min\{d_{\text{edit}}(y_2, \tilde{y}), d_{\text{edit}}(y, \tilde{y}_2)\} & y_1 \neq \tilde{y}_1, \end{cases}$$

with $d_{\text{edit}}(y, \tilde{y}) = \text{len}(y)$ if $\tilde{y}$ is empty and vice versa.

Lemma 2.6. Let $n, m \in \mathbb{N}$ with $n \geq m$, where m is the generation length and n is the watermark key length. Define the decoder Γ by equation (1), alignment cost $d = d_\gamma$ with d_0 from equation (2) and $\gamma > 1/2$, and ϕ by Algorithm 3 using block size $k \leq m$ that divides evenly into m . Let $\xi, \xi' \stackrel{i.i.d.}{\sim} \text{Unif}(\Xi^n)$ with $Y = \text{generate}(\xi; m, p, \Gamma)$. Let $\tilde{Y} \in \mathcal{V}^m$ be conditionally independent of ξ and ξ' given Y , with $d_{\text{edit}}(Y, \tilde{Y}) \leq \varepsilon m$. Then almost surely

$$\mathbb{P}(\phi(\tilde{Y}, \xi') \leq \phi(\tilde{Y}, \xi) \mid \tilde{Y}, Y) \leq mn(2k)^{k/(4\gamma-1)} \exp(-kC_0^2(\alpha(Y) - \gamma\varepsilon)_+^2/2).$$

We prove the result by showing there must exist a length k substring of the corrupted text $\tilde{Y}$ within edit distance $k\varepsilon$ of a substring of Y that the detector will be able to distinguish as watermarked. For fixed k , the set of strings within edit distance εk of an original block watermarked text blows up combinatorially with ε . To ensure we can detect the watermark, the result implies we must set $\gamma = \Omega(1/\varepsilon)$, which means our bound on the expected p -value is vacuous as soon as $\varepsilon = \Omega(1/\log k)$. Admittedly, our analysis is not tight; for example, as a preview of the experimental results to come, in practice we find smaller values of γ (i.e., $\gamma < 1$) to perform significantly better. However, one takeaway from the result is that using a block size $k < m$, where here m is the length of the input text, for detection can be an effective strategy when the user has substantially corrupted the text. The assumption that k divides evenly into m is an artifact of our analysis and not important in practice.

2.4.2 What we run in practice

In practice, to reduce overhead in both **generate** and **detect**, we use a single random permutation⁷ instead of a full sequence, i.e., we let $\pi_i = \pi$ for all $i \in [n]$ for $\pi \sim \text{Unif}(\pi)$. Recall Theorem 1 makes no assumption about the distribution of the permutations; thus, the watermark is still distortion-free. Also, for the test statistic, we find using

$$d(y, (u, \pi)) := \sum_{i=1}^{\text{len}(y)} |u_i - \eta(\pi_i(y_i))| \quad (3)$$

as the alignment cost performs better empirically than the alignment cost in equation (2). To reiterate, the output of **detect** is a valid p -value irrespective of the test statistic we use.

Henceforth, we refer to this version of the watermarking strategy as **ITS**, and we refer to the corresponding Levenshtein version as **ITS-edit**, wherein we define the base alignment cost d_0 by equation (3) and use the following simplified notion of Levenshtein cost:

⁷In principle, with a single random permutation the permuted token indices of both watermarked and nonwatermarked text are no longer conditionally independent of each other, and so the results of Lemmas 2.4, 2.5 and 2.6 no longer apply. However, in practice we observe no degradation in statistical power. Also, irrespective of the lemmas, the p -values from **detect** are still valid by construction.

Definition 5. (simple Levenshtein cost) Let $\gamma \in \mathbb{R}$ and $d_0 : \mathcal{V} \times \Xi \rightarrow \mathbb{R}$. For $y \in \mathcal{V}^*$ and $\xi \in \Xi^*$, define the alignment cost function $d_\gamma : \mathcal{V}^* \times \Xi^* \rightarrow \mathbb{R}$ by

$$d_\gamma(y, \xi) := \min \begin{cases} d_\gamma(y_{2:}, \xi_{2:}) + d_0(y_1, \xi_1) \\ d_\gamma(y, \xi_{2:}) + \gamma \\ d_\gamma(y_{2:}, \xi) + \gamma, \end{cases}$$

with $d_\gamma(y, (u, \pi)) := \gamma \cdot \text{len}(y)$ if ξ is empty and vice versa (as base cases).⁸

In summary, for ITS we use the decoder from equation (1), the test statistic from Algorithm 3 with the alignment cost from equation (3), and the watermark key distribution as the uniform distribution over $[0, 1]^n \times \Pi$, where recall n is the length of the watermark key sequence. Meanwhile, ITS-edit differs from ITS only in that we define the test statistic using the Levenshtein cost from Definition 5 with the base cost again from equation (3).

2.5 Watermarking via exponential minimum sampling

Aaronson [1] proposes mapping variables in $[0, 1]^N$ to tokens in the vocabulary $[N]$ using exponential minimum sampling to generate watermarked text. Whereas Aaronson [1] proposes the use of distortion-inducing hashes much like Kirchenbauer et al. [13], we use exponential minimum sampling to implement the decoder in **generate**, which (after defining a suitable corresponding test statistic) enables an alternative distortion-free and robust watermarking strategy to inverse transform sampling. In particular, for $\xi \in [0, 1]^N =: \Xi$ and $\mu \in \Delta([N])$, define the decoder by

$$\Gamma(\xi, \mu) := \arg \min_{i \in [N]} -\log(\xi_i)/\mu(i). \quad (4)$$

We show this decoder is distortion-free in Theorem 2, whose proof we defer to Appendix C.

Theorem 2. *Define the decoder Γ by equation (4) and let $\xi \sim \text{Unif}([0, 1]^N)$. Then Γ is distortion-free with respect to ξ .*

For the sake of analysis, we define the alignment cost as a slight variation of the proposal of Aaronson [1] (see Section 2.5.2) by

$$d(y, \xi) := - \sum_{i=1}^{\text{len}(y)} \log \xi_{i, y_i}, \quad (5)$$

again defining the test statistic ϕ by Algorithm 3. Similar to Lemma 2.3 for ITS, we exactly characterize the difference in the expected values of the alignment cost on watermarked versus non-watermarked text in terms of the watermark potential of the text. We defer the proof of Lemma 2.7 to Appendix C.

Lemma 2.7. *Let $n \in \mathbb{N}$. Define Γ by equation (4) and d by equation (5). Let $\xi, \xi' \stackrel{i.i.d.}{\sim} \text{Unif}(\Xi^n)$ with $Y = \text{generate}(\xi; n, p, \Gamma)$. Then almost surely for all $i \in [n]$ we have*

$$\mathbb{E}[d(Y_i, \xi'_i) - d(Y_i, \xi_i) \mid Y] = 1 - p(Y_i \mid Y_{i-1}) = \alpha(Y_{i-1:i}).$$

⁸For $y \in \mathcal{V}^*$ (resp., $\xi \in \Xi^*$), we let $y_{\text{len}(y)+1}$: (resp., $\xi_{\text{len}(\xi)+1}$) denote the empty string/sequence.

Summing the result of Lemma 2.7 over $i \in [m]$ implies for any $j \in [n]$ that

$$\mathbb{E}[d(Y, \xi'_{(j+1:j+m)\%n}) - d(Y, \xi_{1:m}) \mid Y] = m\alpha(Y).$$

Thus, defining the test statistic ϕ by Algorithm 3 with respect to the alignment cost d from Eqn (5), we can again upper bound the p -value output by `detect` in Lemma 2.8 using a standard concentration argument and taking a union bound over $j \in [n]$. We defer the proof of Lemma 2.8 to Appendix C. Once again, we actually prove a more general result that allows $\tilde{Y}$ to be any length k subsequence of Y .

Lemma 2.8. *Let $m, n \in \mathbb{N}$ with $n \geq m$. Define Γ by equation (4), d by equation (5), and ϕ by Algorithm 3 with $k = m$. Let $\xi, \xi' \stackrel{i.i.d.}{\sim} \text{Unif}(\Xi^n)$ with $Y = \text{generate}(\xi; n, p, \Gamma)$ and $\tilde{Y} = Y$. Then almost surely*

$$\mathbb{P}(\phi(\tilde{Y}, \xi') \leq \phi(\tilde{Y}, \xi) \mid \tilde{Y}) \leq 2n \exp\left(-\min\{m\alpha(\tilde{Y})^2/8, m\alpha(\tilde{Y})/4\}\right).$$

2.5.1 Robustness to corruptions

Showing high probability p -value upper bounds for corruptions of watermarked text that hold almost surely given the corrupted text—i.e., analogues of Lemmas 2.5 and 2.6—is more difficult, primarily due to the fact that the summands in the alignment metric from equation (5) are no longer bounded and thus bounding the influence of each substitution and/or insertion operation on the test statistic requires more careful analysis. Of course, we could in principle tweak the alignment metric by truncating the summands in order to prove the analogous results; however, as the main intuitions would carry over from Lemmas 2.5 and 2.6 and the results are not critical to the main thrust of the paper, we do not carry this plan out.

2.5.2 What we run in practice

As in the case of ITS, in practice we find using a slight variation of the alignment cost in equation (5) performs better. Namely, following the prescription of Aaronson [1], we modify the previous alignment cost to instead be

$$d(y, \xi) := \sum_{i=1}^k \log(1 - \xi_{i, y_i}). \quad (6)$$

Henceforth, we refer to this version of the watermarking strategy as **EXP**, and we refer to the corresponding Levenshtein version wherein we define the base alignment cost d_0 by equation (6) as **EXP-edit**.

In summary, for **EXP** we use the decoder from equation (4), the test statistic from Algorithm 3 with the alignment cost from equation (6), and the watermark key distribution as the uniform distribution over Ξ^n , where recall n is the length of the watermark key sequence and $\Xi = [0, 1]^N$. Meanwhile, **EXP-edit** differs from **EXP** only in that we define the test statistic using the Levenshtein cost from Definition 5 with the base cost again from equation (6).

3 Experimental results

We empirically validate the statistical power of our watermarking strategies (i.e., ITS, ITS-edit, EXP, and EXP-edit) via experiments with the OPT-1.3B [28] and LLaMA-7B [22] mod-

els.⁹ We run experiments using **generate** rather than **shift-generate**, mainly for the sake of reproducibility; recall however that this choice has no impact on the p -values we report. We test for all watermarks using a block size k (in Algorithm 3) equal to the length m of the text. Following the methodology of Kirchenbauer et al. [13], we generate watermarked text continuations of prompts sampled from the news-like subset of the C4 dataset [16]. We vary the generation length m (Experiment 1) and the random number sequence length n (Experiment 2), and we report median p -values of watermarked text over 500 samples.¹⁰

We also evaluate robustness to four kinds of paraphrasing attacks: randomly substituting a fraction of the generated tokens with tokens chosen uniformly at random from the vocabulary (Experiment 3); randomly inserting a fraction of tokens among the generated tokens (Experiment 4); randomly deleting a fraction of the generated tokens (Experiment 5); using another language model to translate the text from English to French and back (Experiment 6). The first three attacks allow us to systematically vary the level of corruption, while the last attack is an example of an attack we might encounter in the wild. We defer the details of the translation procedures to Appendix D.2.

Finally, using the Alpaca-7B model and evaluation dataset [19], we conduct a case-study on the feasibility of watermarking the responses of a performant instruction-tuned language model to user queries. We also show for certain kinds of instructions that hashing-based watermarks produce noticeably worse responses than our distortion-free watermarks, thus underlining the importance of the distortion-free property in practice.

In all our experiments—except for Experiment 2, where the control variable n is a hyperparameter that is unique to our watermarks—we also replicate the watermark of Kirchenbauer et al. [13] as a baseline, setting the greenlist fraction $\gamma = 0.25$ and varying the logit bias $\delta \in \{1.0, 2.0\}$. We respectively refer to these versions of their watermark as KGW-1.0 and KGW-2.0 after the first three authors’ last names. We emphasize their watermark is not directly comparable to our watermarks as it is not distortion-free (e.g., Kirchenbauer et al. [13] report that even the weakest version we employ with $\delta = 1.0$ and $\gamma = 0.25$ typically increases perplexity by 5–10%).

In their work, Kirchenbauer et al. [13] report approximate p -values, which they obtain from computing the z -score of a certain test statistic. To ensure a fair comparison, we use **detect** (with $T = 5000$) to report p -values for all watermarks;¹¹ in the case of KGW-1.0 and KGW-2.0, we run **detect** using the original inexact p -values they report as the test statistic. We report error bars for the median p -value based on a bootstrapped estimate of the standard deviation using 1000 resamples.

Instead of recomputing the test statistic T times for each prompt—as we originally prescribe in **detect**—to save computation we simply sample T prompts and compute the test statistic once for each ground-truth length m completion; we then use the empirical distribution of these test statistics as the reference distribution within **detect**, which gives a proper p -value with respect to the null hypothesis that the text is an original completion from the dataset. For reference, we include the full pseudocode for this modified version of **detect** in Appendix D.3, and we also plot the full distributions of p -values for nonwatermarked generations (i.e., regular samples from the language models) to verify they are indeed roughly uniform over the interval $[0, 1]$.

⁹We will also at times collectively refer to ITS and ITS-edit as the ITS watermarks and/or strategies and EXP and EXP-edit as the EXP watermarks and/or strategies.

¹⁰The median p -value corresponds to the significance level (i.e., Type I error rate) at which the power of our watermark detector is at least 0.5.

¹¹This setting of T means we never report p -values less than $1/5000$ (i.e., 0.0002) in any of our experiments.